

CO1 AT2

Q15. A plagiarism detection system scans large documents to match a given pattern using brute-force string matching. The document size is very large and pattern occurrences are rare.

- a) Explain how brute-force string matching operates in this scenario.**
- b) Analyze its time complexity.**
- c) Discuss why the approach becomes inefficient for large text inputs.**

a) Explain how brute-force string matching operates in this scenario.

Brute-force string matching compares the given pattern with the document one character at a time. It starts from the first character of the document and checks whether the pattern matches completely. If a mismatch occurs, the pattern is shifted by one position, and the comparison starts again. This process continues until the end of the document is reached or the pattern is found.

In a plagiarism detection system, the algorithm scans every possible position in the large document, even though matching patterns occur very rarely. Therefore, it performs many unnecessary comparisons before concluding whether the pattern exists.

b) Analyze its time complexity.

Let:

n = Length of the document

m = Length of the pattern

Best Case: $O(n)$

When the first character mismatches at every position, only one comparison is made per shift.

Worst Case: $O(n \times m)$

When most characters match before a mismatch occurs, the algorithm compares almost all **m** characters at each of the **(n - m + 1)** positions.

Thus, the worst-case time complexity is:

$$O(n \times m)$$

c) Discuss why the approach becomes inefficient for large text inputs.

Brute-force string matching becomes inefficient for large documents because it checks the pattern at every possible position without using any previously matched information. When the document size is very large, the number of comparisons increases significantly, leading to high execution time. Since pattern occurrences are rare, the algorithm wastes time scanning almost the entire document before determining that the pattern is absent. This makes brute-force string matching slow and unsuitable for plagiarism detection on large datasets. More efficient algorithms such as **Knuth-Morris-Pratt (KMP)** or **Boyer-Moore** reduce unnecessary comparisons and perform much better on large text inputs.

Q18. A text editor searches for keywords in large files using brute-force string matching. Users experience delays as file sizes grow.

- a) Explain how the brute-force string matching algorithm works.**
- b) Analyze the time complexity of this approach.**
- c) Justify why optimized algorithms may be required.**

a) Explain how the brute-force string matching algorithm works.

The brute-force string matching algorithm searches for a keyword by comparing it with the text one character at a time. In a text editor, the algorithm scans every possible position in the file, making repeated comparisons until the keyword is found or the search reaches the end of the document.

b) Analyze the time complexity of this approach.

Let:

- **n** = Length of the text file
- **m** = Length of the keyword (pattern)
- **Best Case: $O(n)$**
This occurs when the first character mismatches at every position, requiring only one comparison per shift.
- **Worst Case: $O(n \times m)$**
This occurs when many characters match before a mismatch happens, causing almost **m** comparisons at each position in the text.

Therefore, the worst-case time complexity is:

$$O(n \times m)$$

c) Justify why optimized algorithms may be required.

As file sizes increase, the brute-force algorithm performs a large number of repeated character comparisons, resulting in noticeable delays. It does not reuse information from previous comparisons and checks every possible position separately, making it inefficient for large documents. Optimized algorithms such as **Knuth-Morris-Pratt (KMP)**, **Boyer-Moore**, or **Rabin-Karp** reduce unnecessary comparisons and search more efficiently. These algorithms provide faster keyword searching, improve the responsiveness of text editors, and are better suited for handling large text files.

Q19. A database maintains sorted employee salary records, allowing Binary Search for quick retrieval. However, frequent updates disturb the sorted order.

a) Explain how Binary Search works when data is sorted.

b) Analyze its time complexity.

c) Discuss the importance of maintaining sorted data for efficient searching.

a) Explain how Binary Search works when data is sorted.

Binary Search is an efficient searching algorithm that works only on **sorted data**. It begins by comparing the target salary with the middle element of the sorted list. If the target matches the middle element, the search is successful. If the target is smaller, the algorithm continues searching only in the left half of the list. If the target is larger, it searches only in the right half. This process is repeated until the required record is found or the search interval becomes empty.

Since the search space is reduced by half after every comparison, Binary Search is much faster than Linear Search for large datasets.

b) Analyze its time complexity.

Let n be the number of employee salary records.

Best Case: $O(1)$

The target salary is found at the middle position in the first comparison.

Worst Case: $O(\log n)$

The search space is halved after each comparison until the element is found or the search ends.

Average Case: $O(\log n)$

Therefore, the time complexity of Binary Search is:

$$O(\log n)$$

c) Discuss the importance of maintaining sorted data for efficient searching.

Binary Search requires the data to be in **sorted order**. If frequent updates such as insertions or deletions disturb the sorting, Binary Search cannot correctly determine which half of the data to search, leading to incorrect results. In such cases, the data must be sorted again before using Binary Search, which adds extra processing time. Therefore, maintaining sorted employee salary records is essential for fast and efficient searching. Properly sorted data ensures that Binary Search can quickly locate records with minimal comparisons, making it highly suitable for large databases.

Q66. A warehouse management system searches for product IDs in an unsorted dataset using Linear Search.

- a) Explain how Linear Search is applied.**
- b) Analyze performance issues.**
- c) Suggest improvements and justify.**

a) Explain how Linear Search is applied.

Linear Search checks each product ID in the dataset one by one until the required product ID is found or the end of the list is reached. The search begins from the first record and compares each product ID with the target ID. If a match is found, the search stops and returns the result. If no match is found after checking all records, the algorithm reports that the product ID is not present.

Since the warehouse data is unsorted, Linear Search is a suitable method because it does not require the records to be arranged in any specific order.

b) Analyze performance issues.

Let **n** be the number of product records.

- **Best Case: $O(1)$**
The required product ID is found in the first record.
- **Average Case: $O(n)$**
On average, about half of the records are checked before finding the product.
- **Worst Case: $O(n)$**
The product ID is located at the last position or is not present, requiring all records to be examined.

As the number of products increases, the search time increases linearly, causing slower performance in large warehouse databases.

c) Suggest improvements and justify.

The performance can be improved by storing the product IDs in **sorted order** and using **Binary Search**, which has a time complexity of **$O(\log n)$** . Another effective improvement is to use a **Hash Table**, which provides an average search time of **$O(1)$** . These methods greatly reduce search time compared to Linear Search and improve the efficiency of the warehouse management system, especially when handling large numbers of product records.

Q101. A weather analysis system collects temperature readings in random order and needs sorting before generating forecasts.

- a) Compare Selection Sort and Bubble Sort for this dataset.**
- b) Analyze their computational cost.**
- c) Recommend the better algorithm with justification.**

a) Compare Selection Sort and Bubble Sort for this dataset.

Selection Sort	Bubble Sort
Finds the smallest element in the unsorted part and places it in the correct position.	Repeatedly compares adjacent elements and swaps them if they are in the wrong order.
Performs fewer swaps.	Performs many swaps, especially when the data is randomly ordered.
Not adaptive to nearly sorted data.	Can stop early with an optimized version if the data becomes sorted.

For randomly ordered temperature readings, both algorithms can sort the data correctly, but Selection Sort generally performs fewer swaps than Bubble Sort.

b) Analyze their computational cost.

Let n be the number of temperature readings.

Selection Sort

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Number of swaps: $O(n)$

Bubble Sort

- Best Case: $O(n)$ (with optimized version)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Number of swaps: Can be $O(n^2)$

Both algorithms have quadratic time complexity in the average and worst cases, making them inefficient for very large datasets.

c) Recommend the better algorithm with justification.

For this weather analysis system, Selection Sort is the better choice because the temperature readings are in random order. It reducing unnecessary data movement and improving efficiency. Although both algorithms have $O(n^2)$ time complexity, Selection Sort is generally faster. However, for very large weather datasets, more efficient algorithms such as Merge Sort or Quick Sort are recommended .

